

Assignment 4

Exercise 1: Ring Communication

Problem: Write an MPI program where each process sends a message to the

next process in a ring topology. Process 0 sends to Process 1, Process 1 sends to Process 2, and the last process sends back to Process 0.

Requirements:

Start with an initial value of 100 in Process 0

Each process adds its rank to the value before sending

Print the value at each process

Final value should return to Process 0

Hint: Use modulo operator to wrap around: $\text{next_rank} = (\text{rank} + 1) \% \text{size}$

Test with: `mpirun -np 4 ./ring_comm`

Observation (Ring Communication)

- Program was executed using multiple MPI processes.
- Process 0 initialized the value as 100.
- The value was passed from one process to the next in a circular (ring) manner.
- Each process added its own rank to the received value before forwarding it.
- Output showed that every process received and updated the value correctly.
- After completing one full round, the final value returned back to process 0.

- For 4 processes, the final value obtained was 106, which is correct.

```
jasjot@DESKTOP-K1LCIOV:~$ nano ringtop.c
jasjot@DESKTOP-K1LCIOV:~$ mpicc ringtop.c -o ring_comm
jasjot@DESKTOP-K1LCIOV:~$ mpirun -n 4 ./ring_comm
Process 1 updated value to 101
Process 2 updated value to 103
Process 0 starting with value 100
Process 3 updated value to 106
Process 0 received final value 106
jasjot@DESKTOP-K1LCIOV:~$
```

Conclusion (Ring Communication)

This experiment helped me understand how processes communicate with each other in MPI using a ring topology.

Each process successfully received data from the previous process, added its rank, and sent it to the next process.

The final value returned to process 0 confirmed that communication between all processes was correct.

This experiment also showed the importance of proper send and receive order to avoid deadlock in MPI programs.

Exercise 2: Parallel Array Sum

Problem: Write an MPI program to compute the sum of an array in parallel.

Divide the array among processes, compute local sums, and combine them.

Requirements:

Create an array of size 100 with values 1 to 100 in Process 0

Distribute array portions using MPI_Scatter

Each process computes sum of its portion

Use MPI_Reduce to compute global sum

Verify result: sum should be 5050

Bonus: Also compute and print the average value.

Test with: `mpirun -np 4 ./array_sum`

Observation (Exercise 2: Parallel Array Sum)

- An array of size 100 was created in process 0 with values from 1 to 100.
- The array was divided equally among all processes using MPI_Scatter.
- Each process calculated the sum of its own part of the array.
- Local sums from all processes were combined using MPI_Reduce.
- The final sum obtained at process 0 was 5050, which is correct.
- The average value of the array was calculated as 50.5.
- All processes worked simultaneously, which reduced the computation load on a single process.

```
jasjot@DESKTOP-K1LCIOV:~$ nano parallel_array_sum.c
jasjot@DESKTOP-K1LCIOV:~$ mpicc parallel_array_sum.c -o array_sum
jasjot@DESKTOP-K1LCIOV:~$ mpirun -n 4 ./array_sum
Global Sum = 5050
Average = 50.50
jasjot@DESKTOP-K1LCIOV:~$
```

Conclusion (Exercise 2: Parallel Array Sum)

This experiment helped me understand how a large array can be processed in parallel using MPI.

By dividing the array among multiple processes, each process performed part of the computation.

The use of `MPI_Scatter` and `MPI_Reduce` made data distribution and result collection easy.

The correct final sum proved that parallel computation was performed successfully.

Exercise 3: Finding Maximum and Minimum

Problem:

Write an MPI program where each process generates random numbers, and the program finds the global maximum and minimum values across all processes.

Requirements:

Each process generates 10 random numbers (0-1000)

Each process finds its local maximum and minimum

Use MPI_Reduce with MPI_MAX and MPI_MIN

Process 0 prints the global maximum and minimum

Also print which process had these values (use MPI_MAXLOC/MPI_MINLOC)

Hint: For MPI_MAXLOC, use a struct with value and rank fields.

Observation

- Each process generated 10 random numbers between 0 and 1000.
- Every process calculated its local maximum and minimum value.
- MPI_Reduce with MPI_MAXLOC and MPI_MINLOC was used.
- Process 0 successfully received the global maximum and minimum values.
- The output also showed the process number from which the max and min values came.
- The program executed correctly using multiple processes.

OUTPUT:

```
jasjot@DESKTOP-K1LCIOV:~$ nano min_max.c
jasjot@DESKTOP-K1LCIOV:~$ mpicc min_max.c -o max_min
jasjot@DESKTOP-K1LCIOV:~$ mpiexec -n 4 ./max_min
Global Maximum = 996 (from process 2)
Global Minimum = 43 (from process 3)
jasjot@DESKTOP-K1LCIOV:~$
```

Conclusion

This experiment helped me understand how collective reduction operations work in MPI.

Using MPI_MAXLOC and MPI_MINLOC, both value and process rank were obtained easily.

The program correctly found the global maximum and minimum from all processes, showing effective parallel computation.

Exercise 4: Parallel Dot Product

Problem:

Compute the dot product of two vectors in parallel.

Requirements:

Vector A = [1, 2, 3, 4, 5, 6, 7, 8]

Vector B = [8, 7, 6, 5, 4, 3, 2, 1]

Distribute vectors using MPI_Scatter

Each process computes partial dot product

Use MPI_Reduce to sum partial products

Expected result: 120

Observation:

- Two vectors A and B of size 8 were used.
- The vectors were divided equally among all processes using MPI_Scatter.
- Each process calculated the dot product of its assigned elements.
- Partial results from all processes were combined using MPI_Reduce.
- The final dot product obtained was 120, which is correct.
- The program executed successfully using multiple processes.

```
jasjot@DESKTOP-K1LCIOV:~$ nano dot_product.c
jasjot@DESKTOP-K1LCIOV:~$ mpicc dot_product.c -o dot_product
jasjot@DESKTOP-K1LCIOV:~$ mpirun -n 4 ./dot_product
Dot Product = 120
jasjot@DESKTOP-K1LCIOV:~$
```

Conclusion:

This experiment helped me understand how vector operations can be performed in parallel using MPI.

By dividing the vectors among multiple processes, computation was done simultaneously.

The correct final result shows that parallel dot product using MPI works efficiently.

SUBMITTED TO : DR. Saif Nalband

SUBMITTED BY: JASJOT SINGH

ROLL NO: 102483032